

HW05

2026-09-27

Question 1

a)

```
set.seed(1)
n <- 400
x0 <- c(0.5, 0.7, 1)
ks <- seq(1, 29, by = 2)
f0 <- 0.5 * x0[1] + sin(x0[2]) - 0.3 * x0[3]^2
f0
```

```
## [1] 0.5942177
```

```
X <- matrix(rnorm(n * 3), ncol = 3)

y <- 0.5 * X[,1] + sin(X[,2]) - 0.3 * X[,3]^2 +
  rnorm(n, 0, 0.1)

d <- sqrt(rowSums((X - matrix(x0, n, 3, byrow = TRUE))^2))
order_d <- order(d)
pred <- sapply(ks, function(k) {
  mean(y[order_d[1:k]])
})

pred
```

```
## [1] 0.1139487 0.6178155 0.5919035 0.6647614 0.5756990 0.6185293 0.6576051
## [8] 0.5582793 0.6250000 0.6371977 0.6252886 0.6002597 0.6272429 0.6168810
## [15] 0.5777179
```

b)

```
set.seed(1)

B <- 200
pred_mat <- matrix(0, nrow = B, ncol = length(ks))

for (b in 1:B) {

  X <- matrix(rnorm(n * 3), ncol = 3)

  y <- 0.5 * X[,1] + sin(X[,2]) - 0.3 * X[,3]^2 +
```

```

    rnorm(n, 0, 0.1)

d <- sqrt(rowSums((X - matrix(x0, n, 3, byrow = TRUE))^2))
order_d <- order(d)

for (j in 1:length(ks)) {
  k <- ks[j]
  pred_mat[b, j] <- mean(y[order_d[1:k]])
}
}

```

Bias:

```
bias <- colMeans(pred_mat) - f0
```

Squared bias:

```
bias2 <- bias^2
```

```

variance <- colMeans(
  (pred_mat - matrix(colMeans(pred_mat),
                      nrow = B,
                      ncol = length(ks),
                      byrow = TRUE))^2
)
mse <- colMeans((pred_mat - f0)^2)
cbind(ks, bias2, variance, mse, bias2 + variance)

```

```

##      ks      bias2      variance      mse
## [1,]  1 0.0003557871 0.039555489 0.03991128 0.03991128
## [2,]  3 0.0002119618 0.018192394 0.01840436 0.01840436
## [3,]  5 0.0016762992 0.014148844 0.01582514 0.01582514
## [4,]  7 0.0025813734 0.011758575 0.01433995 0.01433995
## [5,]  9 0.0030615512 0.011910832 0.01497238 0.01497238
## [6,] 11 0.0032992755 0.011188090 0.01448737 0.01448737
## [7,] 13 0.0039088371 0.010532319 0.01444116 0.01444116
## [8,] 15 0.0040706348 0.008970606 0.01304124 0.01304124
## [9,] 17 0.0052623032 0.007977276 0.01323958 0.01323958
## [10,] 19 0.0050857161 0.007866314 0.01295203 0.01295203
## [11,] 21 0.0061294442 0.007743076 0.01387252 0.01387252
## [12,] 23 0.0065428256 0.007015909 0.01355873 0.01355873
## [13,] 25 0.0082550815 0.006741304 0.01499639 0.01499639
## [14,] 27 0.0098604565 0.006590386 0.01645084 0.01645084
## [15,] 29 0.0108962724 0.005959422 0.01685569 0.01685569

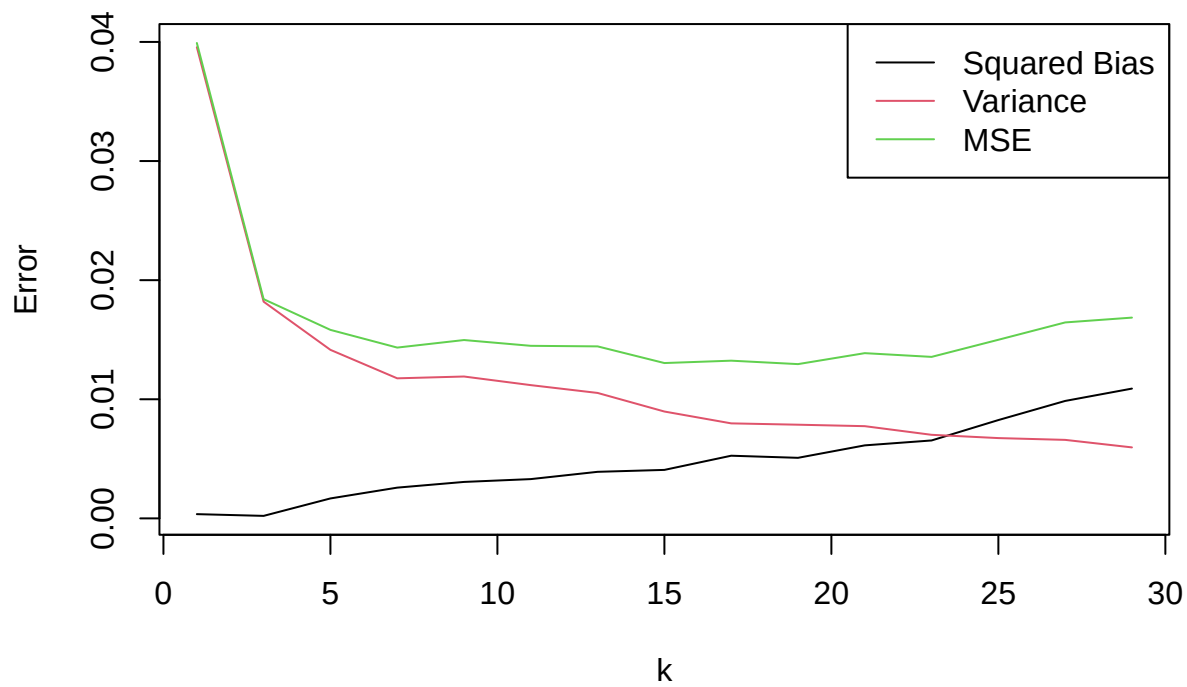
```

```

matplot(
  ks,
  cbind(bias2, variance, mse),
  type = "l",
  lty = 1,
  xlab = "k",
  ylab = "Error"
)

```

```
)
legend(
  "topright",
  legend = c("Squared Bias", "Variance", "MSE"),
  lty = 1,
  col = 1:3
)
```



When k gets larger, variance decreases because more observations are averaged together. At the same time, bias tends to increase because the model becomes less flexible. Thus, small k has lower bias but higher variance, while large k has higher bias but lower variance. The MSE balances these two effects. This is similar to ridge and lasso. More regularization makes the model less flexible, which usually increases bias and decreases variance.

Question 2

a)

```
library(glmnet)
```

```
##      Matrix
```

```
## Loaded glmnet 5.1
```

```

library(MASS)

set.seed(1)

n <- 400
ntest <- 1000
p <- 30
lambda_grid <- 10^seq(-4, 0, length.out = 41)

f <- function(X) {
  0.5 * X[,1] + sin(X[,2]) - 0.3 * X[,3]^2
}

# run KNN and lasso for one setting
run_setting <- function(correlated = FALSE) {

  # ----- generate data -----
  if (!correlated) {
    Xtrain <- matrix(rnorm(n * p), n, p)
    Xtest <- matrix(rnorm(ntest * p), ntest, p)
  } else {
    Sigma <- matrix(0.8, p, p)
    diag(Sigma) <- 1

    Xtrain <- mvrnorm(n, rep(0, p), Sigma)
    Xtest <- mvrnorm(ntest, rep(0, p), Sigma)
  }

  ytrain <- f(Xtrain) + rnorm(n, 0, 0.1)
  ytest <- f(Xtest) + rnorm(ntest, 0, 0.1)

  # ----- 5NN -----
  knn_pred <- numeric(ntest)

  for (i in 1:ntest) {
    d <- sqrt(rowSums((Xtrain -
      matrix(Xtest[i,], n, p, byrow = TRUE))^2))

    nearest <- order(d)[1:5]
    knn_pred[i] <- mean(ytrain[nearest])
  }

  knn_mse <- mean((ytest - knn_pred)^2)

  # ----- 10-fold CV for lasso -----
  folds <- sample(rep(1:10, length.out = n))
  cv_mse <- numeric(length(lambda_grid))

  for (j in 1:length(lambda_grid)) {

    fold_error <- numeric(10)

```

```

for (fold in 1:10) {

  train_id <- folds != fold
  val_id <- folds == fold

  # means and SDs from training part only
  xmean <- colMeans(Xtrain[train_id,])
  xsd <- sqrt(colMeans(
    (Xtrain[train_id,] -
      matrix(xmean, sum(train_id), p, byrow = TRUE))^2
  ))

  ymean <- mean(ytrain[train_id])

  Xtr <- sweep(Xtrain[train_id,], 2, xmean, "-")
  Xtr <- sweep(Xtr, 2, xsd, "/")

  Xval <- sweep(Xtrain[val_id,], 2, xmean, "-")
  Xval <- sweep(Xval, 2, xsd, "/")

  ytr <- ytrain[train_id] - ymean

  fit <- glmnet(
    Xtr, ytr,
    alpha = 1,
    lambda = lambda_grid[j],
    standardize = FALSE,
    intercept = FALSE
  )

  pred <- as.numeric(predict(fit, Xval)) + ymean

  fold_error[fold] <- mean(
    (ytrain[val_id] - pred)^2
  )
}

cv_mse[j] <- mean(fold_error)
}

best_lambda <- lambda_grid[which.min(cv_mse)]

# ----- refit lasso using all training data -----
xmean <- colMeans(Xtrain)
xsd <- sqrt(colMeans(
  (Xtrain - matrix(xmean, n, p, byrow = TRUE))^2
))

ymean <- mean(ytrain)

Xtr <- sweep(Xtrain, 2, xmean, "-")
Xtr <- sweep(Xtr, 2, xsd, "/")

```

```

Xte <- sweep(Xtest, 2, xmean, "-")
Xte <- sweep(Xte, 2, xsd, "/")

fit <- glmnet(
  Xtr, ytrain - ymean,
  alpha = 1,
  lambda = best_lambda,
  standardize = FALSE,
  intercept = FALSE
)

lasso_pred <- as.numeric(predict(fit, Xte)) + ymean
lasso_mse <- mean((ytest - lasso_pred)^2)

beta <- as.numeric(coef(fit))[-1]
nonzero <- sum(abs(beta) > 1e-8)

c(KNN_MSE = knn_mse,
  Lasso_MSE = lasso_mse,
  Lambda = best_lambda,
  Nonzero = nonzero)
}

# Setting 1 and Setting 2
setting1 <- run_setting(FALSE)
setting2 <- run_setting(TRUE)

results <- data.frame(
  Setting = c("Independent", "Correlated"),
  KNN_MSE = c(setting1["KNN_MSE"], setting2["KNN_MSE"]),
  Lasso_MSE = c(setting1["Lasso_MSE"], setting2["Lasso_MSE"]),
  Lambda = c(setting1["Lambda"], setting2["Lambda"]),
  Nonzero = c(setting1["Nonzero"], setting2["Nonzero"])
)

results

```

```

##      Setting  KNN_MSE Lasso_MSE   Lambda Nonzero
## 1 Independent 0.520289 0.2419825 0.05011872      3
## 2 Correlated 0.172453 0.2139940 0.01584893      8

```

b)

The results are different between the independent and correlated settings because correlation changes the meaning of Euclidean distance. When the variables are highly correlated, observations that are close in the 30-dimensional space may have different neighborhood structures than in the independent case. KNN uses all 30 variables when calculating distance, including variables that do not directly affect the response. In contrast, lasso can shrink some coefficients to zero, so it does not use all covariates equally.

c)

The true response contains nonlinear terms, but the lasso model contains only linear terms. Therefore, lasso cannot represent the true response function exactly. KNN does not assume a linear relationship, so it can capture nonlinear patterns locally.

However, the results from these two datasets do not show that correlation always helps KNN or that one method is always better. Performance depends on the data structure, the number of variables, the correlation, and whether the assumed model matches the true relationship.

Question 3

a)

```
set.seed(1)

n <- 200
ntest <- 200
p <- 100
ks <- seq(2, 82, by = 8)

run_one <- function(m) {

  A <- matrix(runif(p * m, -2, 2), p, m)

  Ztrain <- matrix(rnorm(n * m), n, m)
  Ztest <- matrix(rnorm(ntest * m), ntest, m)

  Xtrain <- Ztrain %*% t(A) + matrix(rnorm(n * p), n, p)
  Xtest <- Ztest %*% t(A) + matrix(rnorm(ntest * p), ntest, p)

  ytrain <- Ztrain[,1] + Ztrain[,2] + rnorm(n)
  ytest <- Ztest[,1] + Ztest[,2] + rnorm(ntest)

  mse <- numeric(length(ks))

  for (j in 1:length(ks)) {
    k <- ks[j]
    pred <- numeric(ntest)

    for (i in 1:ntest) {
      d <- sqrt(rowSums((Xtrain - Xtest[i,])^2))
      id <- order(d)[1:k]
      pred[i] <- mean(ytrain[id])
    }

    mse[j] <- mean((ytest - pred)^2)
  }

  mse
}

mse3 <- run_one(3)
mse30 <- run_one(30)

data.frame(
  k = ks,
```

```

MSE_m3 = mse3,
MSE_m30 = mse30
)

```

```

##      k  MSE_m3  MSE_m30
## 1    2 4.176387 4.762556
## 2   10 3.002598 2.777013
## 3   18 2.994334 2.744869
## 4   26 2.985678 2.700987
## 5   34 3.005226 2.675634
## 6   42 3.028693 2.640824
## 7   50 3.003422 2.644982
## 8   58 2.973663 2.663271
## 9   66 2.986680 2.684412
## 10  74 2.995806 2.669812
## 11  82 3.017081 2.668466

```

The table shows the test MSE for each value of k when $m=3$ and $m=30$.
The same training and test data are used for all values of k within each setting.

b)

```

set.seed(1)

B <- 50

all_mse3 <- matrix(0, B, length(ks))
all_mse30 <- matrix(0, B, length(ks))

for (b in 1:B) {
  all_mse3[b,] <- run_one(3)
  all_mse30[b,] <- run_one(30)
}

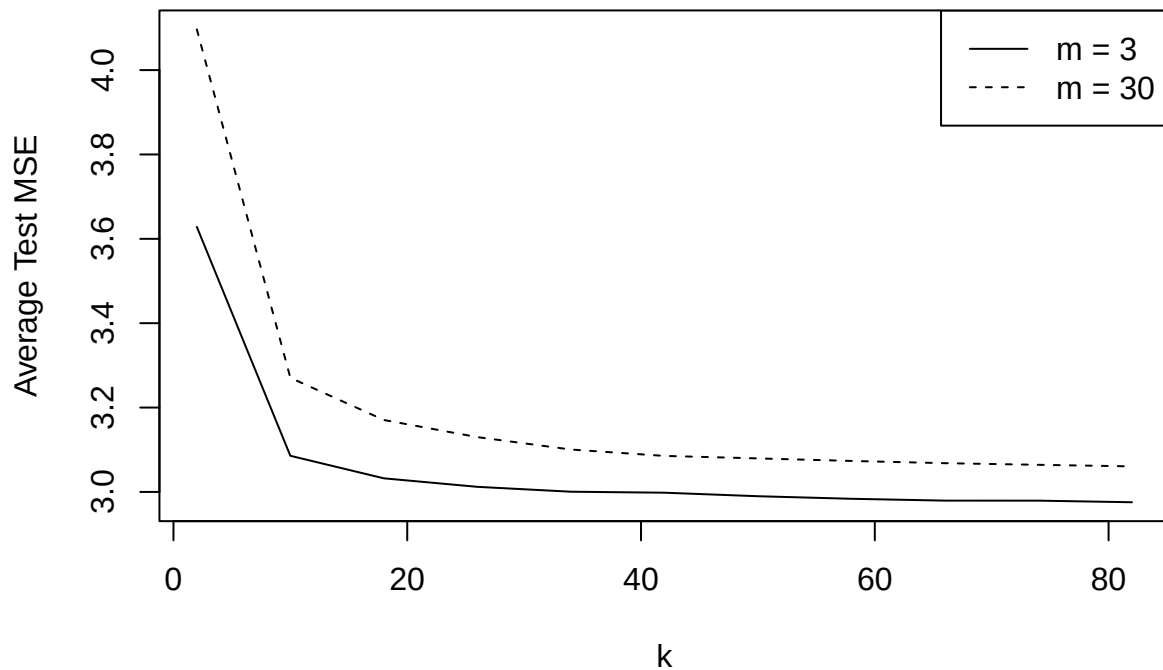
avg3 <- colMeans(all_mse3)
avg30 <- colMeans(all_mse30)

plot(ks, avg3, type = "l",
     xlab = "k", ylab = "Average Test MSE",
     ylim = range(c(avg3, avg30)))

lines(ks, avg30, lty = 2)

legend("topright",
     legend = c("m = 3", "m = 30"),
     lty = c(1, 2))

```



The average test MSE changes with k for both settings.

The two curves can be different even though both use 100 observed variables. This is because the hidden structure of the data is different when $m=3$ and $m=30$.

c)

KNN uses distance between the observed X values.

When $m=3$, the data depend on fewer latent variables, so the observations have a simpler structure.

When $m=30$, the structure is more complex, so finding useful nearest neighbors can be harder.

The noise η_i adds random variation to every observed variable. Because of this noise, the observed X values are not exactly on an m -dimensional linear space. So even though both settings have 100 observed variables and Y depends only on two latent variables, their KNN performance can still be different.

Question 4

```
train <- read.csv("data/knn/zip-train.csv", header = FALSE)
test  <- read.csv("data/knn/zip-test.csv", header = FALSE)

dim(train)
```

```
## [1] 7291 257
```

```
dim(test)
```

```
## [1] 2007 257
```

```

train <- train[1:1000, ]
test  <- test[1:500, ]

ytrain <- train[, 1]
Xtrain <- as.matrix(train[, -1])

ytest  <- test[, 1]
Xtest  <- as.matrix(test[, -1])

```

a)

```

knn5 <- function(x0, Xtrain, ytrain, method) {

  diff <- sweep(Xtrain, 2, x0, "-")

  if (method == "euclidean") {
    d <- sqrt(rowSums(diff^2))
  } else {
    d <- rowSums(abs(diff))
  }

  nearest <- order(d, 1:nrow(Xtrain))[1:5]

  votes <- table(factor(ytrain[nearest], levels = 0:9))

  which.max(votes) - 1
}

```

I wrote a 5NN classifier that can use either Euclidean or Manhattan distance. It finds the five nearest training images and uses the most common digit as the prediction. Distance ties are broken by the smaller training row number, and voting ties are broken by the smaller digit.

b)

```

pred_E <- numeric(500)
pred_M <- numeric(500)

for (i in 1:500) {
  pred_E[i] <- knn5(Xtest[i, ], Xtrain, ytrain, "euclidean")
  pred_M[i] <- knn5(Xtest[i, ], Xtrain, ytrain, "manhattan")
}

error_E <- mean(pred_E != ytest)
error_M <- mean(pred_M != ytest)

error_E

```

```
## [1] 0.122
```

```
error_M
```

```
## [1] 0.138
```

```

conf_E <- table(
  Predicted = factor(pred_E, levels = 0:9),
  True = factor(ytest, levels = 0:9)
)

conf_M <- table(
  Predicted = factor(pred_M, levels = 0:9),
  True = factor(ytest, levels = 0:9)
)

conf_E

```

```

##           True
## Predicted  0   1   2   3   4   5   6   7   8   9
##           0 118   0   7   1   0   4   2   0   2   0
##           1   0  66   0   0   2   0   0   0   0   1
##           2   0   0  39   1   0   0   2   0   0   1
##           3   0   0   0  23   0   4   0   0   2   0
##           4   0   0   1   0  27   1   1   1   0   1
##           5   0   0   0   1   0  13   0   0   0   0
##           6   0   2   0   0   0   1  41   0   2   0
##           7   0   0   0   0   1   1   0  34   0   4
##           8   0   0   3   2   0   2   0   0  37   0
##           9   1   0   0   1   4   0   0   1   1  41

```

```

conf_M

```

```

##           True
## Predicted  0   1   2   3   4   5   6   7   8   9
##           0 118   0   6   1   0   5   3   0   4   0
##           1   0  67   2   0   2   0   1   0   0   1
##           2   0   0  35   0   0   0   2   0   0   0
##           3   0   0   1  24   0   4   0   0   2   0
##           4   0   0   1   1  27   1   0   1   0   1
##           5   0   0   0   1   0  11   0   0   0   0
##           6   0   1   0   0   0   2  40   0   2   0
##           7   0   0   2   1   1   0   0  34   0   5
##           8   0   0   2   0   0   3   0   0  34   0
##           9   1   0   1   1   4   0   0   1   2  41

```

The test classification error was 0.122 for Euclidean distance and 0.138 for Manhattan distance. Therefore, Euclidean distance performed slightly better on this test set.

c)

```

wrong <- which(pred_E != ytest)

par(mfrow = c(1, 3))

for (i in wrong[1:3]) {

```

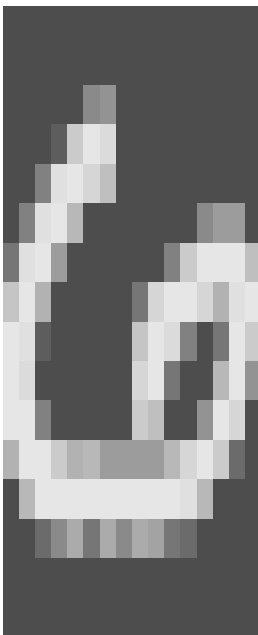
```

img <- matrix(Xtest[i, ], 16, 16, byrow = TRUE)

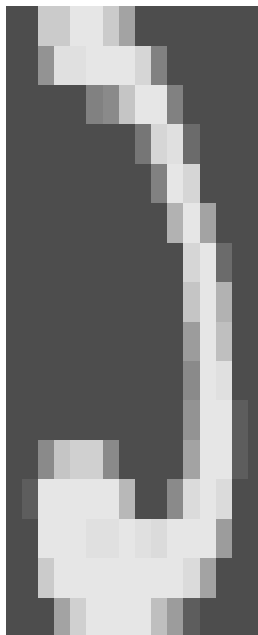
image(
  t(img[16:1, ]),
  col = gray.colors(20),
  axes = FALSE,
  main = paste("True:", ytest[i],
               "Pred:", pred_E[i])
)
}

```

True: 6 Pred: 0



True: 2 Pred: 0



True: 4 Pred: 7



```

par(mfrow = c(1, 1))

wrong_E <- which(pred_E != ytest)
wrong_M <- which(pred_M != ytest)

length(wrong_E)

```

```
## [1] 61
```

```
length(wrong_M)
```

```
## [1] 69
```

```
length(intersect(wrong_E, wrong_M))
```

```
## [1] 58
```

```
length(setdiff(wrong_E, wrong_M))
```

```
## [1] 3
```

```
length(setdiff(wrong_M, wrong_E))
```

```
## [1] 11
```

The mistakes are understandable because some handwritten digits have similar shapes or are not very clear. Euclidean distance made 61 mistakes, and Manhattan distance made 69 mistakes. They made 58 of the same mistakes, but they also made some different mistakes.

This shows that the two methods can have similar error rates but still classify some images differently because they measure distance in different ways.

Question 5

STAT 432 Plot Formatting

Description

Use this skill only when I explicitly ask to improve the formatting of a STAT 432 homework plot. It should improve the appearance of plots made in R or Python without changing the data or statistical meaning.

Rules

- Use reasonable margins so titles, labels, and legends are not cut off.
- Add a clear and short title when appropriate.
- Use simple and descriptive x-axis and y-axis labels.
- Use colors that are easy to distinguish and avoid too many colors.
- Make text, points, and lines large enough to read clearly.
- Keep the plot size balanced so the figure is easy to view in homework.
- Do not add unnecessary decorations or change the statistical content of the plot.